CSC343H5Y **Side A**

Introduction to Databases (Lecture 8 – Functional Dependencies & Normalization)

Functional Dependencies (FDs)

Definition. $X \to Y$ holds in R: whenever two tuples of R agree on all attributes of X, they must also agree on all attributes of Y (X *functionally determines* Y). X, Y, Z denote sets of attributes; write ABC for $\{A, B, C\}$. FDs come from domain knowledge – the DBMS cannot discover them, only enforce them.

Trivial FDs. $X \to Y$ is trivial if $Y \subseteq X$ (e.g. $A \to A$, $ABC \to A$). Always true, never useful.

Splitting / combining (RHS only). $X \to A_1 A_2 \cdots A_n$ holds iff $X \to A_i$ holds for every i. So $A \to BC$ splits into $A \to B$ and $A \to C$. *No such rule for the LHS.*

Armstrong's Axioms.

- **Reflexivity:** if $X \supseteq Y$ then $X \to Y$.
- **Augmentation:** if $X \to Y$ then $XZ \to YZ$ for any Z.
- **Transitivity:** if $X \to Y$ and $Y \to Z$ then $X \to Z$.

FDs generalize keys: $X \to R$ (all attributes) means X is a **superkey**.

Attribute Closure (Y^+)

Y^+ w.r.t. FD set F: all attributes derivable from Y using F.

1. Start: current set $= Y$.
2. Scan every FD $X \to A$ in F. If $X \subseteq$ current set, add A.
3. Repeat until nothing new is added.

If $Y^+ =$ all attributes of R, then Y is a **superkey**; if additionally no subset of Y is a superkey, Y is a **candidate key**.

Example. $R(a, b, c, d, e, f)$, $F = \{ab \to c,\ ac \to d,\ c \to e,\ ade \to f\}$. Find $\{a, b\}^+$: $\{a, b\} \xrightarrow{ab \to c} \{a, b, c\} \xrightarrow{ac \to d} \{a, b, c, d\} \xrightarrow{c \to e} \{a, b, c, d, e\} \xrightarrow{ade \to f} \{a, b, c, d, e, f\}$. So $\{a, b\}^+ = \{a, b, c, d, e, f\}$ – ab is a superkey (here, a candidate key).

Minimal Basis (Minimal Cover)

A minimal basis F' of F satisfies: (1) every RHS is a single attribute; (2) no FD can be removed (all necessary); (3) no attribute can be removed from any LHS.

Construction.

1. Split every RHS into singletons.
2. For each FD f: temporarily remove it; if the remaining set still entails f (check via closure), leave it removed.
3. For each attribute A on a LHS: temporarily remove A; if the FD still holds (closure of the smaller LHS still contains the RHS), leave it removed.
4. Repeat steps 2–3 until no more changes.

Example. $F = \{A \to AC,\ B \to ABC,\ D \to ABC\}$. Split RHS: $\{A \to A,\ A \to C,\ B \to A,\ B \to B,\ B \to C,\ D \to A,\ D \to B,\ D \to C\}$. Drop trivial ($A \to A$, $B \to B$). $B \to C$ is redundant since $B \to A$ and $A \to C$ give it by transitivity; drop it. $D \to A$ and $D \to C$ are redundant since $D \to B \to A$ and $D \to B \to A \to C$; drop them. $\Rightarrow F' = \{A \to C,\ B \to A,\ D \to B\}$.

Schema Decomposition

Split R into $R_1, \ldots, R_n$ ($R_i \subset R$, no new attributes) and split F into $F_1, \ldots, F_n$ (each F_i uses only attributes of R_i; F entails every F_i).

Lossless join test (binary decomposition). $R \to R_1, R_2$ is lossless iff

$$(R_1 \cap R_2) \to R_1 \ \in F^+ \quad \text{or} \quad (R_1 \cap R_2) \to R_2 \ \in F^+.$$

TLDR: lossless iff the shared column(s) form a key in **at least one** of the two new relations. Lossy decompositions produce **extra (spurious) tuples** when rejoined.

FD projection onto R_i. For every subset $X \subseteq R_i$: compute X^+ using F; for every $A \in X^+ \cap R_i$, add $X \to A$ to F_i. Then reduce F_i to a minimal basis. (Ignore trivial FDs and supersets of an X with $X^+ = R$ – expensive in general, exponential in $|R_i|$.)

Goals of a good decomposition: lossless join, dependency preservation ($((F_1 \cup \cdots \cup F_n)^+ = F^+$), and anomaly-free (no redundant data).

Normal Forms

Hierarchy: BCNF $\subseteq$ 3NF $\subseteq$ 2NF $\subseteq$ 1NF.

1NF	No multi-valued attributes (no lists in a cell).
2NF	Every non-prime attribute depends on the *whole* candidate key (not part of it).
3NF	For every non-trivial $X \to A$: X is a superkey **or** A is prime (part of some key).
BCNF	For every non-trivial $X \to A$: X **must** be a superkey (no exception for prime A).

BCNF decomposition algorithm. While some FD $X \to Y$ violates BCNF (X not a superkey): compute X^+; replace R by $R_1 = X^+$ and $R_2 = R - (X^+ - X)$ (i.e. $X \cup (R - X^+)$); recurse on R_1, R_2, projecting F onto each. This decomposition is always lossless ($R_1 \cap R_2 = X$, and $X \to R_1$).

BCNF vs. 3NF. BCNF: lossless & anomaly-free, but may **not** preserve all dependencies. 3NF: lossless & dependency-preserving, but may still allow **some** anomalies (redundancy from FDs whose RHS is prime).

Use Side A as a reference. Schema: ***Movies(title, year, star, studio, studioAddress, salary)****. FDs:* $F = \{$ *title, year* $\rightarrow$ *studio*, *studio* $\rightarrow$ *studioAddress*, *star* $\rightarrow$ *salary* $\}$.

Box 1 Attribute Closure & Keys *(6 min)*

(a) Compute $\{\text{title, year}\}^+$ using F. Show your work step by step.

(b) Is {title, year, star} a candidate key of `Movies`? Justify using your closure from (a).

. . . . . .

. . . . . .

(c) Is `studio` $\rightarrow$ `studioAddress` a trivial FD? Why or why not?

. . . . . .

Box 2 Minimal Basis *(8 min, discuss)*

Consider $G = \{\, AB \rightarrow CD,\ \ C \rightarrow A,\ \ D \rightarrow B \,\}$ over relation $R(A, B, C, D)$.
(a) First split every right-hand side into singletons. Write the resulting set.

. . . . . .

(b) Check each FD for redundancy (can it be derived from the others via closure?). Circle any FD you can remove, and briefly justify one of your removals.

(c) Check each remaining left-hand side for extra attributes (e.g. can $AB \rightarrow C$ be shrunk to $A \rightarrow C$ or $B \rightarrow C$?). Give the final **minimal basis**.

. . . . . .

. . . . . .

Box 3 Decomposition, Losslessness & BCNF *(8 min)*

A classmate proposes decomposing `Movies` into:
`R1(title, year, star, salary)` and `R2(title, year, studio, studioAddress)`.
(a) What is $R_1 \cap R_2$? Using the lossless-join test on Side A, is this decomposition lossless? Show which FD (if any) justifies your answer.

. . . . . .

. . . . . .

(b) Is `R2(title, year, studio, studioAddress)` in BCNF with respect to its projected FDs? If not, identify the violating FD and give a lossless BCNF decomposition of `R2`.

(c) Is your decomposition from (b) still in **3NF** if instead we only required 3NF? Would the original (undecomposed) `R2` already satisfy 3NF? Explain in one sentence.

. . . . . .

. . . . . .